

CloudServe Solutions

Intelligent customer support triage

Forward Deployed AI Engineering capstone project report Prepared for review · 20 September 2026
Author: Mouli

This report describes the implementation in this repository and the saved evidence available on the preparation date. It distinguishes working software, operational benchmark proxies, and outcomes that require customer or independent reviewer evidence. It does not certify production readiness.

Reading guide

1. Executive summary
2. The problem
3. Discovery findings
4. Requirements
5. Architecture and design
6. Implementation
7. Evaluation
8. Governance and risk
9. The requirements revision
10. Conclusions

Appendices contain the evidence register, recording and reproduction commands, and submission notes. The companion `architecture_diagram.html` provides an interactive view of the implemented workflow.

Evidence boundary. The latest saved model-backed result covers all eighty validation inputs. This report uses the retained aggregate and paired per-ticket artifact for that run. Offline mode remains a separate demonstration and testing capability; its results must not be combined with the model-backed score.

1. Executive summary

CloudServe's stated request was a chatbot. The underlying need was faster, more dependable support: make existing documentation accessible, avoid repeating work during escalation, and preserve accountability when automation cannot answer safely. The supplied development data supports that distinction. Of 500 tickets, 357 are labelled answerable from documentation, but only 311 are labelled suitable for automatic response. Knowing an answer and being authorised to send it are different decisions. [S1, S4]

The implementation provides a shared pipeline for batch evaluation and a FastAPI interface. It normalises four channel formats, retrieves passages from a 29-document knowledge base, classifies intent and urgency, applies explicit routing policies, drafts cited responses, checks selected safety conditions, and persists decisions in SQLite. Escalations include context, sources, and a reason. Dataset labels remain outside runtime decision-making and are used for evaluation only. [S6]

The strongest demonstrated result is the ability to process and account for a complete supplied evaluation batch. The saved model-backed validation run contains 56 automatic responses and 24 escalations across 80 inputs, with 80 reconciled decisions and no guardrail blocks. It records 88.51% weighted intent precision, 70% automation, and 12.287 seconds P95 processing latency. All seven selected operational benchmarks pass. These are observations on the supplied validation set under its recorded configuration, not an estimate of production performance. [S7]

The console retains seven operational KPIs. Its first-contact-resolution label is explicitly an automation proxy; citation accuracy measures retrieved-document-ID validity; its latency comparison uses a legacy 300-second benchmark. The framework's stricter three-second requirement and actual customer outcomes remain separate. A passing operational comparison is therefore not acceptance of every project requirement. [S3, S9]

The most important caveat is that independently verified answer quality has not been established. The grounding reviewer is disabled in the current runtime. Citation checks still run, but a valid citation does not prove that every claim is supported. The next release should prioritise reviewed quality evidence, classification and policy error analysis, and a frozen, reproducible evaluation before autonomous customer delivery.

2. The problem

2.1 From a chatbot request to a support outcome

The brief describes a growing business whose support processes have not kept pace with its customer base. More than 500 tickets arrive each week. First reply takes eight to twelve hours, first-contact resolution is 42%, and customer satisfaction is 3.2 out of five. These are supplied case-study baselines, not measurements collected from a live CloudServe deployment during this project. [S1]

The visible symptom is a slow reply. The underlying workflow contains several sources of delay: interpreting the question, searching scattered documentation, deciding whether an answer is safe, writing it, and reconstructing context when another engineer takes over. Putting a conversational interface in front of that workflow would not by itself remove these delays. It could also hide uncertainty behind confident language.

The project therefore treats support as a sequence of decisions. A ticket must first become a consistent input. Relevant knowledge must be found. The system must determine whether the request falls inside the automation boundary. Only then should it draft an answer. If any of those conditions fails, the useful output is a handover that reduces the next person's work.

This framing changes the definition of success. Producing more text is not the goal. A useful automatic reply must be correct, appropriate to the customer's request, and within policy. A useful escalation must explain what is known and what remains uncertain. Both outcomes need a reconstructable decision record.

Table 1 separates the supplied business baseline from the assessment targets. The initial client aspiration of 65% first-contact resolution and satisfaction of 4.2 is stronger than the minimum evaluation targets of 60% and 4.0. Keeping these distinct avoids silently changing the meaning of success.

Table 1. Supplied business context and minimum evaluation targets. Source: Project Brief, sections 1 and 7.

Measure	Supplied baseline	Evaluation target
First-contact resolution	42%	At least 60%
First reply	8–12 hours	Mean under 5 minutes
Satisfaction	3.2 / 5	At least 4.0 / 5
Escalation	58%	At most 30%
Repeat contacts	No headline baseline	Reduce by half

2.2 Users, scope, and the value of escalation

The supplied discovery material identifies different needs across support leadership, first-line agents, specialist engineers, documentation owners, and customers. Leadership needs outcomes that can be defended. First-line agents need reliable access to existing answers. Specialist engineers need escalations that preserve context. Documentation owners need traceable sources. Customers need useful answers without repeating their problem. These needs favour a shared support workflow over an isolated chat experience. [S4]

Four input channels are represented: email, chat, documentation comments, and community forums. The implementation accepts these channel types in structured payloads. It does not connect to a mail server, chat product, documentation platform, or forum service. Channel integration is a deployment task beyond the implemented input adapter.

The automation boundary is equally important. Routine, sufficiently confident, documented requests can proceed to generation. Security incidents, compliance and residency requests, selected financial and deletion requests, enterprise high-urgency cases, unclear requests, low-confidence predictions, and missing documentation can require a human. The exact rule precedence is documented in section 5 and implemented in `src/route.py`.

An escalation is not automatically an error. Some are required by policy; others indicate missing information or weak model performance. Reporting only a total escalation rate combines these causes and makes improvement harder. The evaluation must distinguish correct escalation, avoidable escalation, and unsafe automatic response.

The current output is a response draft or a structured escalation payload returned by the API or harness. There is no customer delivery connector, support queue assignment, closure tracking, or follow-up conversation. Consequently, the implementation can demonstrate processing behaviour and decision traceability, but cannot yet demonstrate reduced customer waiting time, confirmed resolution, or reduced repeat contacts.

For a pilot, the useful first deployment would place these outputs in front of agents, collect their corrections and disposition decisions, and measure whether the handover actually saves work. That proposal is future work. The report does not claim an agent review interface or customer pilot has already been delivered.

3. Discovery findings

3.1 Documentation coverage creates an opportunity, not a guarantee

The Stage 1 workbook records 357 of 500 development tickets as answerable from existing documentation: 71.4%. That is strong evidence for improving knowledge access. It is not evidence that a model resolves 71.4% of tickets, because the field is a reference label about the documentation rather than an observed system outcome. [S4]

Only 311 development tickets, or 62.2%, are labelled for automatic response. A further 189 require escalation. The difference between documentation answerability and allowed automation is central to the design. A request can have relevant documentation while still involving an account-specific decision, a security issue, or another policy exclusion.

The corpus contains 29 documentation articles. The system indexes that approved material rather than treating every past answer as authoritative. The 200 supplied reference responses are useful for comparison and review, but the current harness does not implement an independent quality assessment against them. A reference collection should not be described as an active evaluator merely because it exists in the repository.

Table 2 summarises the opportunity and its boundaries. The percentages use the 500-ticket development denominator. The groups overlap: a high-urgency ticket can also be answerable from documentation and require human handling. Adding these percentages would therefore be misleading.

Table 2. Development dataset findings used in design. Source: development labels and Stage 1 workbook.

Finding	Count / 500	Implication
Documentation-answerable	357 / 500, 71.4%	Retrieve approved knowledge
Expected automatic response	311 / 500, 62.2%	Preserve an eligibility boundary
Expected escalation	189 / 500, 37.8%	Make handover a first-class output
Must not auto-respond	87 / 500, 17.4%	Test prohibited automation explicitly
High urgency	146 / 500, 29.2%	Measure urgency recall separately

The engineering consequence is a separation between retrieval, policy routing, generation, and validation. Retrieval answers whether the system found relevant material. Routing answers whether automation is appropriate. Generation constructs a response. Validation checks selected properties of that response. No one component can stand in for all four decisions.

3.2 Workflow friction and channel diversity

The supplied interview evidence, summarised in the discovery workbook, points to time spent finding an answer and reconstructing context rather than a shortage of all possible answers. First-line work includes interpreting the issue, searching documentation or snippets, composing a reply, and deciding whether to escalate. Specialist work can repeat the interpretation step when the handover contains little more than the original ticket. [S4]

This finding produced two complementary outputs: cited drafts for eligible requests and context-rich escalation packets for the rest. The latter preserve classification, confidence, policy flags, attempted retrieval, and customer context. The intention is to reduce repeated investigation. Actual time saved by engineers has not been measured, so the report treats that as a hypothesis for a pilot rather than a completed benefit.

The channel mix also argues against designing for only short chat questions. Email represents 212 development inputs, or 42.4%; chat represents 155, or 31.0%; documentation comments represent 78, or 15.6%; forums represent 55, or 11.0%. Normalisation allows these formats to share retrieval and routing logic without duplicating the downstream pipeline. [S4]

There are 22 intent labels, with no single category dominating. Data export and data residency each contain 29 examples; rate limiting contains 13. Even the larger classes are small samples for evaluating a model. A high weighted score can conceal poor performance on a smaller category, particularly if that category has a high consequence when misrouted.

The design therefore retains per-class precision, recall, support, and confusion matrices in the detailed evaluation. It also records routing disagreements separately from classification mistakes. A wrong intent can still produce a conservative escalation, while a correct intent can be paired with an inappropriate routing policy.

An important implementation lesson follows: a handover should preserve evidence rather than manufacture certainty. A low-confidence prediction remains low confidence after it is placed inside a structured packet. Similarly, retrieved documents should be presented as candidate support, not as proof that a human engineer will find them sufficient. The packet makes the decision inspectable; it does not eliminate the need for judgement.

3.3 Baselines, fairness, and conflicting objectives

The development history records first-contact resolution for 219 of 500 tickets, or 43.8%, and repeat contact for 108, or 21.6%. These describe the supplied historical workflow. They cannot be reused as the outcomes of the newly implemented system. The workbook also records average historical satisfaction of 2.97 out of five, below the brief's headline 3.2. The difference is a reminder that a sample and an organisational baseline are not interchangeable. [S4]

Non-fluent customers represent 120 development tickets, or 24%. The workbook reports historical first-contact resolution of 45.8% for this group, which does not support a simple claim that this sample demonstrates worse aggregate resolution for non-fluent customers. Interview concerns still matter, but they need testing rather than repetition as established fact.

The system retains tier, region, and fluency metadata for subgroup analysis. Comparing classification accuracy across these groups is useful diagnostic evidence. It is not the same as comparing independently reviewed response quality, and it cannot establish that a deployment treats customers fairly. A group can receive accurate classifications and still receive unhelpful drafts or unnecessary escalation.

There is also a measurable tension between the reference routing mix and the business target. The development labels expect 37.8% escalation and the validation labels expect 40%. Both exceed the stated target of at most 30%. A system following those labels perfectly would still miss the aggregate escalation target on those datasets. The right response is to examine the eligible population and business assumptions, not to override safety labels to make the score green.

This finding should inform the next PRD revision. Report total escalation alongside policy-required and avoidable escalation, preserve the safety constraints, and agree how the business target applies to the eligible population. Until stakeholders resolve that definition, the report should show the tension explicitly.

Finally, validation has already been inspected and used during troubleshooting. It can still reveal defects, but it no longer supports a claim of a pristine, once-only unseen evaluation. A fresh holdout or the assessor's hidden set is needed for that claim. The exact historical number of validation runs is not reconstructable from the retained latest files; this uncertainty is declared rather than replaced with a guessed count.

4. Requirements

4.1 Functional traceability

The Stage 2 PRD turns the discovery findings into twelve functional requirements. Table 3 connects those identifiers to the implemented components and identifies the relevant boundary. This is an implementation trace, not a declaration that all external acceptance tests have passed. [S5, S6]

Table 3. Functional requirements and implementation coverage.

ID	Requirement	Implementation and boundary
FR-01	Ingest multiple channels	<code>ingest.py</code> ; payload formats, no external connectors
FR-02	Predict intent and urgency	<code>classify.py</code> ; confidence is not calibrated probability
FR-03	Retrieve documentation	<code>retrieve.py</code> ; Chroma search with relevance floor
FR-04	Route deterministically	<code>route.py</code> ; explicit policy precedence
FR-05	Exclude high-risk requests	Intent and content rules; semantic coverage incomplete
FR-06	Supply useful handover	Structured packet; no external queue dispatch
FR-07	Draft cited responses	<code>generate.py</code> ; claim support not independently proven
FR-08	Validate before release	<code>guardrails.py</code> ; grounding reviewer disabled
FR-09	Persist decisions	SQLite log plus run-scoped reconciliation
FR-10	Evaluate unattended	Harness CLI, aggregate and per-ticket JSON
FR-11	Handle failures	Per-record batch recovery; audit failures remain failures
FR-12	Halt automation	Kill switch at routing; process-local API setting

FR-03 follows from documentation coverage and search friction. FR-06 follows from the repeated work described in escalation. FR-05 and FR-08 follow from the cost of an inappropriate or unsafe reply. FR-09 supports accountability across both successful responses and failures. These relationships form the trace from discovery to implementation rather than treating the PRD as a separate paperwork exercise.

The functional boundary needs precise wording. “Auto-respond” is the route name used by the code; in this implementation it means an accepted draft is returned. “Escalate” means a packet is created and returned. Neither term demonstrates that an external customer or engineer received anything. An integration requirement must explicitly add delivery, acknowledgement, and failure handling before those outcomes can be claimed.

The supplied build specification remains the acceptance authority. This report documents coverage and evidence limits; it does not replace the twelve acceptance criteria with the seven console KPIs.

4.2 Non-functional requirements and evidence

The non-functional requirements combine latency, quality, privacy, auditability, fairness, operating cost, calibration, reproducibility, and testability. These need different forms of evidence. A unit test can confirm that a block suppresses a draft; it cannot establish customer satisfaction or service availability. [S2, S3, S5]

Table 4. Non-functional requirements: current evidence and remaining work.

ID	Requirement area	Evidence or remaining work
NFR-01	P95 latency under 3 seconds	Latest 80-input P95 is 12.287 s; scope is all processed inputs
NFR-02	Availability at least 99.5%	No uptime observation window established
NFR-03	Intent and response quality	Weighted precision measured; independent response review pending
NFR-04	No outbound private data	Selected regex checks; broader/manual review pending
NFR-05	Exact audit coverage	Latest run reconciles 80 of 80 input positions
NFR-06	Group quality variation under 5 pp	Classification diagnostics available; reviewed quality pending
NFR-07	Cost and efficiency	Local retrieval; no proven operating cost or response cache
NFR-08	Confidence calibration within 5 pp	Diagnostics exist; calibration acceptance not established
NFR-09	Reproducible clean setup	Locked environment; independent cold-machine check pending
NFR-10	Testability	Component and integration tests; no claim of production load coverage

Table 4 intentionally separates implementation from proof. The run-scoped audit result is strong evidence about those eighty records. It says nothing about a future database outage. Conversely, a missing customer satisfaction measure does not mean the pipeline is broken; it means the measurement requires customer or assessor participation not provided by batch processing.

The selected console benchmark uses weighted intent precision and a 300-second P95 comparison. Those definitions are retained for continuity with the requested console layout. The framework's per-class quality expectations and three-second latency requirement are not changed by this presentation choice. Any proposed acceptance change needs an explicit rationale, evidence, and stakeholder decision.

For the next revision, acceptance should specify population, execution mode, warm-up treatment, denominator, reviewer method, and evidence artifact. This makes it possible to decide whether a requirement passed without interpreting a convenient proxy after the run. It also prevents a future report from treating an absent measurement as either a failure of implementation or a fabricated success.

5. Architecture and design

5.1 The implemented control flow

Both the API and the harness call `process_ticket` in `src/graph.py`. Normalisation happens before the compiled graph. The implemented graph retrieves before classification, then applies routing. Figure 1 shows the actual node order, rather than reproducing the illustrative order in the brief. [S6]



Figure 1. Runtime control flow. A guardrail block changes the route to escalation and suppresses the draft; all normal terminal paths proceed through logging.

The shared state carries ticket context, retrieved passages, predictions, routing reasons, generation output, guardrail findings, and audit identifiers. That explicit state makes intermediate decisions inspectable and allows component tests to supply controlled inputs. The graph has a short, bounded path; it does not implement an open-ended autonomous agent loop.

The route node is the policy boundary. An escalation bypasses generation. An eligible request proceeds to generation and then the guardrail node. The latter can preserve the automatic route or replace it with a blocked escalation. Logging follows either result before the API or harness receives the final state.

The kill switch is evaluated at routing, after retrieval and classification. It halts automatic responses but does not stop all upstream computation or provider calls. Moving a global emergency check earlier is a reasonable future improvement; drawing it at ingress would misrepresent the current implementation.

The interactive HTML diagram contains the same control flow, component details, and illustrative route traces. It runs locally without a server or external assets. Its highlighted scenarios explain the design; they are not replayed production events and do not call a model.

5.2 Knowledge, models, and persistence

The knowledge preparation path loads the 29 supplied articles, splits them into chunks of 700 characters with 100-character overlap, embeds them using `all-MiniLM-L6-v2`, and persists them in Chroma. These are character counts, not token counts. At runtime, retrieval selects up to three passages above the configured relevance threshold. The environment template specifies 0.40; the saved full validation run explicitly records 0.30. Reproducing that result therefore requires the recorded setting rather than assuming the template matches. [S6]

Local embeddings and storage keep the small corpus straightforward to operate. Retrieval evidence is retained with the decision so a reviewer can see which material was available. A retrieved passage can still be irrelevant or incomplete; the threshold is a design setting requiring measurement, not a proof of support.

Classification and generation use OpenRouter with the configured `meta-llama/llama-3.1-8b-instruct` model. These are separate responsibilities and can each affect latency and quality. The classifier returns an intent, urgency, alternatives, and a numeric confidence. The current number is model-reported confidence, not the output of an independently fitted calibration procedure.

Offline mode uses a heuristic classifier and an extractive response fallback. The fallback quotes a retrieved passage with an AI-draft disclosure. It is useful for reproducible pipeline tests, but it is a different decision system from model-backed execution. Its latency and classification scores must not be advertised as the hosted model's performance.

SQLite stores decisions, including the final action, reasons, sources, guardrail results, and available handover context. The evaluation path attaches a run identifier and input position. Reconciliation compares the expected and stored records for that run, rather than counting every ticket that has ever appeared in the database.

There are two distinct data boundaries. First, evaluation labels and historical outcomes remain outside runtime inference. They enter scoring after the decision is produced. Second, model-backed operation involves an external model provider. Local vector search does not make the entire system local. Before real customer use, prompt contents, data handling, access controls, and retention need an explicit operational review. The present report documents this boundary without claiming that a deployment agreement or production control has already been established.

5.3 Routing, guardrails, and alternatives

The router applies ordered checks: kill switch; high-risk intent; content-policy exclusions and feature requests; enterprise high urgency; unclear request; classification confidence below 0.80; and missing relevant documentation. If none applies, the request becomes eligible for generation. The first applicable reason determines the escalation explanation. [S6]

This ordering gives explicit policy precedence over confidence. A confident security classification does not make a security incident eligible. Content rules also cover some mixed-intent requests where the primary class looks routine. Their limitation is that regular expressions can be too broad or miss semantic variations. The exposed-key example in section 7 demonstrates why policy false positives need review.

The response gate checks selected PII and secret patterns, citation presence and membership in the retrieved sources, prompt-injection patterns, unauthorised commitments, generation confidence, and missing or unusable responses. It suppresses a blocked draft and creates an escalation. The semantic grounding reviewer is disabled at the user's direction; the retained reviewer module is not part of the live decision path.

Table 5. Design choices and their trade-offs.

Choice	Why it fits this project	Trade-off
Bounded LangGraph workflow	Explicit branches and shared state	More structure than one prompt
Local Chroma retrieval	Small approved corpus; reproducible index	Index freshness remains an operating task
Deterministic policy routing	Explainable precedence and reasons	Regex exclusions need careful error analysis
SQLite decision store	Simple persistent audit for a prototype	Concurrency and operations need future work
Offline fallback	Tests and demonstrations without provider dependence	Different quality and timing characteristics
Reviewer disabled	Avoids strict reviewer determining routes	Semantic faithfulness remains unverified

Table 5 describes the rationale and alternatives, not a claim that every alternative was experimentally benchmarked. A single prompt deciding everything would be simpler, but harder to isolate and audit. A fully distributed service architecture would add operational overhead without addressing the current quality gaps. The selected architecture is appropriate for a local capstone prototype; authenticated administration, service-level observability, and external delivery remain separate engineering work.

6. Implementation

6.1 Modules and reproducible execution

The implementation is organised around responsibilities rather than channel-specific copies of the workflow. `ingest.py` normalises input. `retrieve.py` manages the index and search. `classify.py` supplies intent and urgency. `route.py` applies policy. `generate.py` produces drafts. `guardrails.py` validates them. `logging_store.py` persists decisions, and `graph.py` composes the path. `api.py` and the evaluation harness are entry points to that shared logic. [S6]

The current dependency configuration specifies Python 3.14. The lockfile and environment template make configuration explicit. A local index must be prepared before meaningful retrieval. Offline mode avoids model-provider calls, but first-time embedding model downloads and dependency installation may still require connectivity. “Offline” describes execution with available local dependencies and model weights, not a guarantee that a completely empty machine can bootstrap without a network.

The harness accepts an input path and an output directory. The directory form writes fixed names, `latest_report.json` and `latest_ticket_results.json`. Each ticket produces a processing message, and the final console report groups volume, business, technical, and governance results before the seven KPI rows. An explicit report path and ticket-output path are available when a run must be preserved.

The API exposes ticket processing, a database-oriented health check, operational counts, and kill-switch controls. Its `/metrics` endpoint returns application JSON rather than Prometheus exposition. `/health` does not establish model-provider or Chroma readiness. The source does not start a server merely by invoking the API module; the demonstrated server command should use `uvicorn src.api:app`.

The prompt library separates classification and generation instructions. Classification supplies structured predictions; generation asks for source-supported, cited answers. Prompt requests are design controls, not guarantees. Their outputs still need parsing, policy checks, and evaluation. The retained grounding prompt is diagnostic material while runtime review is disabled.

Reproducibility also depends on preserving evidence. Reusing the latest output directory replaces earlier results. For a final gate, named artifacts should be retained together with run identifiers, dataset hash, source and prompt hashes, configuration, and execution mode. A future report update can then point to one frozen release rather than a changing latest file.

6.2 Reliability, audit recovery, and technical debt

Three implementation details materially affect the trustworthiness of the evaluation. First, runtime input is restricted to operational ticket fields. Reference labels and historical outcomes are not passed into graph inference. This prevents a labelled evaluation dataset from giving the router information it would not have in deployment.

Second, batch processing handles errors per input record. An invalid record or processing exception can become a logged escalation and the batch can continue. If the audit write itself fails, the run cannot claim complete coverage: the record is marked accordingly and the harness reports an incomplete audit with a failing exit status. Continuing the batch is not the same as declaring it successful.

Third, reconciliation is scoped to the current run and input position. A repeated ticket identifier at two positions is two inputs and needs two decisions. Missing, extra, and duplicate records must not be hidden by a set of unique ticket IDs or by old rows from previous runs. This makes the audit result more meaningful than an all-time database count.

Automated tests cover components, graph behaviour, API responses, routing exclusions, guardrail suppression, batch recovery, and metric definitions. During the preceding implementation review, the suite reached 120 passed and two skipped after runtime grounding review was disabled. Subsequent inline-reporting changes passed 27 focused metric and batch-recovery checks. The calculations were also compared with the prior implementation using the 80 saved ticket executions, with matching metric fields. These are regression checks, not a fresh full-suite result or proof of model quality.

The most useful restructuring would make configuration and provider behaviour more explicit, move an emergency automation stop ahead of unnecessary upstream work, and separate processing latency from actual customer delivery latency. Administration also needs authentication and a shared configuration mechanism before multi-worker deployment. The current kill-switch API changes a process environment variable; it is not a demonstrated distributed control plane.

Further work should add durable external handover and delivery acknowledgements, stronger secret and PII coverage, structured provider failure telemetry, and operational readiness checks. None requires replacing the bounded workflow. They require completing the interfaces around it and proving the resulting behaviour under failure, load, and realistic reviewed tickets.

7. Evaluation

7.1 Method and provenance

The evaluation separates dataset analysis, deterministic software checks, offline batch execution, model-backed execution, and independent quality review. Each answers a different question. Dataset labels describe expected behaviour. Unit tests check implemented rules. Offline runs exercise a fallback pipeline. Model-backed runs observe provider-dependent predictions and drafts. Human review is needed for the specified claim-support and response-quality evidence. [S3]

Table 6 identifies the retained evidence used in this report. The latest model-backed aggregate uses schema 2.1 and the operational-proxy profile. Its eighteen recorded source and prompt hashes identify the evaluated snapshot. Reporting functions were subsequently consolidated into the standalone harness; the saved run has not been rerun for that structural change. The saved retrieval threshold is 0.30, routing confidence is 0.80, and generation confidence floor is 0.70. No new model run was launched to write this report.

Table 6. Retained evaluation artifacts cited in this report.

Evidence	Dataset and mode	Scope and observed outcome
latest_report.json	Validation; model enabled	80 inputs; 56 auto; 24 escalate; 80 reconciled
latest_ticket_results.json	Same 80-input run	Predictions, labels, sources, reasons, drafts and review status

The model-backed run ID is RUN-a674be199f52485d92240e373cc74fc5, timestamped 20 September 2026 at 15:28:01 UTC. Its source dataset hash is recorded in the ticket artifact. The complete validation file contains 80 inputs and all 80 were processed. The run records no invalid inputs or processing errors.

Offline execution remains useful for controlled demonstrations and regression testing. Its heuristic classification and extractive generation differ from the hosted model path. Any recorded offline batch should therefore retain its own artifacts and be reported separately. In particular, cached local processing latency is not comparable to a model-provider round trip, and an offline completion does not establish hosted-model quality.

Validation was used more than once during troubleshooting and its contents have been examined. The single-use validation condition is therefore not satisfied by describing the latest artifact as a once-only unseen test. Older console excerpts are historical evidence, but cannot substitute for a preserved aggregate, per-ticket results, and matching source snapshot. A fresh protected evaluation is the appropriate next acceptance step after development decisions are frozen.

7.2 Results: business, technical, and governance

On the latest 80-input model-backed run, 56 drafts were accepted for the automatic route and 24 inputs escalated. The automation proxy is therefore 70% and escalation is 30%. No customer delivery or closure data accompanies these records. There is no supported claim that 70% of customers were resolved on first contact, or that satisfaction improved. [S7]

Table 7 preserves the selected seven-KPI console interpretation. It uses saved measurements, not a new run. The operational score is seven passes out of seven under these definitions. A zero detection count refers to scanner findings, while citation validity refers to source IDs. Neither establishes complete privacy or factual correctness.

Table 7. Seven operational KPI comparisons from the saved 80-input validation result.

KPI	Value	Selected comparison	Result
01 FCR: automation proxy	70.00%	At least 60%	Pass
02 Escalation rate	30.00%	At most 30%	Pass
03 P95 processing latency	12.287 s	Under 300 s, legacy	Pass
04 Weighted intent precision	88.51%	At least 85%	Pass
05 Draft PII detections	0	Zero scanner findings	Pass
06 Exact audit reconciliation	80 / 80, 100%	Exact 100%	Pass
07 Citation ID validity	100%	At least 95%	Pass

Technical diagnostics add context: 68 of 80 intents were correct, retrieval found an expected source on 51 of 53 evaluable tickets, and mean processing time was 5.070 seconds. P95 uses nearest-rank selection and includes processing through logging. The observed 12.287 seconds exceeds the framework's three-second benchmark for this measured scope. The document-mismatch hallucination proxy flags 3 of 56 automatic responses, or 5.36%; this is a heuristic, not a human-reviewed hallucination measurement.

Governance evidence is strongest for audit accounting: every input position in the run has a matching decision. There were no guardrail blocks, and 56 automatic drafts record grounding review as disabled and not run. The absence of blocks is not proof that every draft was safe; it means none triggered the enabled checks in this sample.

Independent citation faithfulness, human-reviewed hallucination, customer satisfaction, confirmed resolution, repeat-contact reduction, and availability have not been established. These are stated as evidence gaps here rather than added as extra rows to the user's selected seven-KPI console table.

7.3 Error analysis and interpretation

The 80-input run is useful for diagnosis beyond the seven passing benchmarks. Its labels expect 48 automatic responses and 32 escalations. Actual routing agrees on 58 of 80 inputs, or 72.5%, with seven unnecessary escalations relative to labels and fifteen automatic responses where escalation was expected. None of the fourteen explicit must-not-auto-respond cases was automated, but that narrower success does not eliminate the other routing errors. This is more actionable than concluding that every increase in escalation is a report corruption. [S7]

One concrete example is VAL-0001, titled “Need to rotate an exposed key.” The classifier predicted `api_key_issue` with 0.95 confidence, while the content-policy rule produced `POLICY_SECURITY_EXPOSURE` and escalated. The reference label expects an automatic response. That disagreement is attributable to the explicit policy boundary, not to the disabled grounding reviewer. The policy may be defensible, too broad, or in conflict with the label interpretation; it requires a documented product decision and development examples before changing it.

Other errors arise before the guardrail stage. Only two of six unclear requests were correctly classified, while high-urgency recall is five of twenty-five, or 20%. Weighted precision conceals these weaknesses. The confidence bin has mean reported confidence of 96.81% and observed intent accuracy of 85%, a gap of 11.81 percentage points. The current confidence floor is therefore not evidence of calibrated reliability.

The saved full run uses a retrieval threshold of 0.30 instead of the 0.40 template value. This accompanies a 96.23% expected-source hit rate, but fourteen inputs labelled not answerable from documentation nevertheless receive automatic drafts. The configuration change cannot be judged from retrieval coverage alone. The right experiment measures expected-source recall, irrelevant-source inclusion, routing changes, and reviewed answer quality together on development data. It should not optimise only the automation percentage.

The disabled grounding reviewer cannot explain blocks in this latest run: the recorded block count is zero. Earlier review-enabled runs and older console reports used different behaviour and should not be mixed with this snapshot. Removing the reviewer eliminates that source of rejection but does not establish that every accepted answer is factually grounded.

The next evaluation should use paired comparisons on a fixed development set, changing one component at a time. Preserve per-ticket reasons and compare route disagreements, class errors, retrieval misses, and independently reviewed draft defects. Freeze the resulting settings before a fresh holdout. This process makes metric changes explainable and avoids choosing thresholds simply because they recreate an earlier favourable console result.

8. Governance and risk

8.1 Controls and residual risk

Governance is implemented partly in the route and response gate, and partly in the audit record. It is not complete merely because those components exist. Table 8 identifies remaining risks and proposes accountable roles for a pilot. These are proposed responsibilities, not recorded stakeholder sign-offs. [S3, S6]

Table 8. Risk register for the implemented prototype and a proposed pilot.

Risk	Likelihood / impact	Current mitigation	Residual work and proposed owner
Unsupported answer with valid citation	Medium / high	Retrieval, cited prompt, ID checks	Independent claim review; support quality lead
Private data or secret in a draft	Medium / high	Selected regex patterns; suppress on detection	Broader/manual tests and retention controls; security lead
Unsafe or unnecessary automation	Medium / high	Policy order and confidence floor	Mixed-intent and policy error review; support owner
Unequal response quality	Unknown / high	Group metadata and diagnostics	Reviewed subgroup samples; support quality lead
Missing decision records	Low in saved run / high	Persistent writes, exact reconciliation	Failure and operational recovery tests; engineering owner
Provider or index failure	Medium / medium	Fallback and batch recovery	Readiness and service monitoring; engineering owner
Unauthorised admin action	Medium if exposed / high	Local prototype usage	Authentication and shared control; engineering owner

The “block and escalate” behaviour is an important implemented control. A detected unsafe draft is suppressed rather than edited and sent. The human receives a reason and context. However, regex checks are only as comprehensive as their patterns. Zero scanner findings should not be described as a verified absence of all private data.

The external model-provider boundary requires attention before using real customer data. A local database and local embedding model do not remove the information transmitted during classification and generation. The pilot needs a clear data minimisation and retention decision, access restrictions, and an approved provider arrangement.

The audit store improves traceability but is not demonstrated to be tamper-proof. SQLite access, backups, retention, and deletion procedures remain operating responsibilities. Logs can themselves contain sensitive customer context, so broad access to the database would undermine the privacy controls applied to responses.

8.2 Fairness, incidents, and operational boundaries

Fairness requires a defined outcome and adequate samples. The harness can compare classification and routing diagnostics across tier, region, and language fluency. These comparisons help locate problems. The required cross-group response-quality claim needs independently reviewed outputs and reported sample counts. No such completed quality audit is claimed here. In the saved 80-input run, fluent customers have 86.89% intent accuracy (n=61) and non-fluent customers 78.95% (n=19), a 7.94-point diagnostic gap. Tier accuracy ranges from 78.57% for standard (n=42) to 100% for enterprise (n=8). These small, unadjusted groups warrant review rather than a causal bias claim.

The proposed audit should sample both accepted drafts and escalations, stratify by the relevant groups, and assess correctness, support, completeness, and appropriateness of routing. Reviewing only accepted responses would miss a system that gives one group less access to automation. Small groups should be reported with uncertainty rather than assigned a confident pass based on a handful of examples.

For a suspected unsafe reply, the immediate operating procedure is to stop automatic responses, preserve the relevant ticket and decision identifiers, inspect the retrieved sources and enabled guardrail findings, and route the case to the support and engineering owners. The local API kill switch supports the first step within its process. In a multi-worker deployment, operators must also stop automation across every worker or delivery integration; that global control is not implemented.

After containment, the engineer should reproduce the issue using a sanitised development fixture, determine whether the cause is policy, retrieval, generation, validation, or delivery, and add a focused regression check. A support owner should review the corrected response and any affected cases. Resuming automation should require evidence that the identified failure is contained, not merely a restarted service.

The API currently lacks authentication for administrative controls. Its health response checks database connectivity and cannot certify end-to-end readiness. There is no demonstrated alerting dashboard, uptime window, incident response rota, or production rollback procedure. The incident process above is a proposed operating runbook built around current capabilities, with those limitations made explicit.

An agent-assisted pilot is the appropriate next boundary: draft generation and handover can be evaluated without asserting autonomous customer resolution. Actual delivery should follow only after independent quality review, access controls, complete audit handling, and a documented release decision. The report does not invent the required customer or stakeholder approvals.

9. The requirements revision

The revision should record what was initially assumed, what the evidence showed, and how that changes the requirement or its acceptance method. It should not simply list code edits or lower a failed target.

Table 9 provides the proposed evidence-led revision narrative. The existing Stage 5 workbook has not been edited as part of these deliverables. [S5, S7]

Table 9. Proposed PRD revisions for review, not approved workbook changes.

Initial assumption	Evidence or observation	Proposed requirement revision
An accepted automatic draft represents FCR	No delivery or closure evidence exists	Separate automation from confirmed resolution; define closure events
Valid source IDs demonstrate grounded answers	ID checks cannot detect unsupported claims; reviewer disabled	Define independent claim-support review and an explicit release criterion
A 0.80 confidence floor is defensible by itself	Model confidence is not calibrated; errors persist	Require development calibration and threshold trade-off evidence
Total escalation at most 30% aligns with labels	Labels require 37.8% dev and 40% validation escalation	Separate required and avoidable escalation; seek target interpretation
A total log count proves complete coverage	Old rows and repeated IDs can conceal gaps	Require exact run/input reconciliation, including failure cases
One latency number proves responsiveness	Offline and hosted modes differ; delivery unmeasured	Specify mode, population, warm-up and separate delivery timestamps

Several revisions strengthen measurement rather than relaxing quality. For example, the three-second latency target remains a requirement even though the selected console comparison uses 300 seconds. The appropriate response to 12.287 seconds on the saved validation run is profiling and a clear measurement scope, not renaming it a full acceptance pass.

The grounding decision is an explicit current deviation: semantic review no longer determines automatic response or escalation. The report and diagram show that state. A future reviewer, if considered, should first be evaluated against independently reviewed examples, including false rejection rates. Reintroducing a strict evaluator without that calibration could again reduce useful automation without establishing better customer outcomes.

The workbook also needs consistency review for requirement identifiers, claimed approvals, threshold history, and revision counts. The effort log needs its daily and stage totals reconciled by the author. Neither actual hours nor stakeholder approval can be inferred from code, so this report leaves those as author-owned corrections rather than fabricating a signed history.

10. Conclusions

The project delivers a coherent end-to-end prototype: one shared workflow handles ingestion, retrieval, classification, policy routing, cited drafting, selected safety checks, and persistent decision records. Batch recovery and exact audit reconciliation make the resulting decisions inspectable. These are meaningful engineering outcomes even where the business and quality targets are not yet demonstrated.

The evidence does not support a production-readiness claim. The latest saved model-backed result covers eighty validation tickets that have already been used during troubleshooting, and independent response-quality review is pending. The current operational scorecard is useful for continuity and diagnosis, provided its proxies remain explicit. A favourable ID-validity result cannot stand in for claim support, and an automatic draft cannot stand in for customer resolution.

The next work should proceed in dependency order. First, freeze the definitions and preserve a complete evidence bundle. Second, analyse development errors in ambiguous classification, policy exclusions, and retrieval misses. Third, independently review drafts and escalations, including subgroup coverage. Fourth, profile hosted execution against the real latency requirement. Finally, add authenticated administration, global automation control, and acknowledged delivery before a customer-facing pilot.

For submission, the report and script provide a consistent account of the current system. The remaining author tasks are to reconcile the PRD revision and effort workbook, record the video with real execution evidence, verify the repository on an independent clean machine, and package the required folders. A fresh protected evaluation is needed if the submission is to claim unseen performance; repeated validation troubleshooting must be disclosed.

AI tool-use declaration. OpenAI Codex assisted with repository review, implementation and evaluation revisions discussed during this project, and preparation of this report, the video script, and architecture HTML. The project author is responsible for checking the resulting code, factual statements, evidence, and final submission. This declaration describes the assistance visible in the current working session; the author should add any earlier tools or substantial external code before submission.

The practical lesson is that the requirements evolve when measurement reveals an assumption. The system becomes more useful when those assumptions are exposed: what counts as resolution, which escalations are required, what a confidence score means, and which safety properties have actually been tested. The next iteration should use that evidence to improve behaviour while preserving transparent reporting.

Appendix A. Evidence register

Table A1. Sources used for this report. Paths are relative to the repository root.

Ref	Source	Use
S1	docs1/01_Read_First/01_Project_Brief.docx	Business context, targets, project scope
S2	docs1/01_Read_First/02_Build_Specification.docx	Acceptance authority
S3	docs1/03_Reference/Evaluation_Framework.docx; Governance_Framework.docx	Measurement and governance obligations
S4	docs/Stage_1_Discovery_Workbook_Filled_Draft.docx; data/development_tickets.json	Discovery findings and dataset counts
S5	docs/Stage_2_PRD_Filled.docx; Stage_5_PRD_Revision_Log.docx	Requirement IDs and revision context
S6	src/; prompts/; tests/; .env.example	Implemented behaviour and checks
S7	evaluation/results/latest_report.json; latest_ticket_results.json	Eighty-input saved validation evidence
S8	data/development_tickets.json; data/documentation.json	Development and knowledge-base inputs; not a separate scored run
S9	evaluation/harness.py	Inline seven-KPI definitions, calculations, and output behaviour
S10	docs1/04_Submission/Submission_Guide.docx	Report, recording and packaging requirements

The evidence files are retained in their existing locations. This document does not rewrite their metrics schema or replace saved measurements. If `latest_report.json` is overwritten later, the run ID and preparation date in section 7 remain necessary for interpreting this report. Preserve a copy of the referenced artifacts with the final submission.

Supporting workbooks remain separate submission artifacts. They include discovery, the PRD, prompt library, sprint plan, revision log, and effort log. Their presence does not imply that all statements are consistent with the current code. The revision and effort discrepancies identified in section 9 should be reconciled before packaging.

Appendix B. Reproduction and submission notes

Run commands from the repository root with the configured environment. The following development commands do not rerun validation or replace the canonical latest evidence.

```
uv sync --frozen --python 3.14
Copy-Item .env.example .env
# Configure the local .env privately; never show its contents on video.
uv run python -m src.retrieve --index
uv run pytest
uv run python -m evaluation.harness --input data/development_tickets.json --offline --output
evaluation/results/video_development
uv run uvicorn src.api:app --host 127.0.0.1 --port 8000
```

Only copy the environment template when creating a fresh checkout; do not overwrite an existing local configuration. The recording output directory uses the fixed filenames inside a separate directory. The full development demonstration is explicitly offline. A recorded offline completion must not be described as the hidden evaluation gate or a new model-backed validation pass.

Table B1. Required submission layout and remaining author actions.

Folder	Required content	Remaining action
01_Video	MP4 or link file	Record 18–22 minutes; at least 7 minutes real demonstration
02_Report	Single report PDF	Review this report and use enrolled full-name filename
03_Workbooks	Five stage workbooks and effort log	Reconcile revisions and actual hours; export required effort PDF
04_Source_Code	Complete repository	Verify clean setup; exclude secrets and irrelevant local artifacts

The guide prescribes `FirstnameLastname_Capstone_Report.pdf`, `FirstnameLastname_Capstone_Video.mp4`, and `FirstnameLastname_Capstone_Submission.zip`, using the author's enrolled full name without spaces. The generated working files use neutral names until that exact spelling is confirmed. No deadline extension or completed upload is asserted here.

The main report contains twenty pages after the cover and before these appendices. Tables and the control-flow figure are numbered and referenced in the text. The video script includes screen actions, narration, real-ticket demonstrations, and a separately labelled controlled guardrail test. Recording, final evidence verification, and submission remain author actions.